TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

PURWANCHAL CAMPUS

A PROJECT ON MACHINE UNLEARNING SDK AND API PLATFORM FOR PRIVACY-COMPLIANT SYSTEM

BY

DEBENDRA SHEKHAR GUPTA (PUR078BCT030)

DIPESH ACHARYA (PUR078BCT032)

JEEVAN KUMAR KUSHWAHA (PUR078BCT037)

PRIYANKA KUMARI MISHRA (PUR078BCT057)

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

PURWANCHAL CAMPUS

DHARAN, NEPAL

March 31, 2026

A Machine Unlearning SDK and API Platform for Privacy-compliant ML Systems

BY

DEBENDRA SHEKHAR GUPTA (PUR078BCT030)

DIPESH ACHARYA (PUR078BCT032)

JEEVAN KUMAR KUSHWAHA (PUR078BCT037)

PRIYANKA KUMARI MISHRA (PUR078BCT057)

Project Supervisor

Asst. Prof. Pukar Karki

A project submitted to the Department of Electronics and Computer Engineering in partial fulfillment of the requirements for the Bachelor's Degree in Computer Engineering

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

PURWANCHAL CAMPUS,INSTITUTE OF ENGINEERING

TRIBHUVAN UNIVERSITY

DHARAN, NEPAL

March 31, 2026

COPYRIGHT ©

DECLARATION

We declare that the work hereby submitted for the degree of Bachelor of Engineering in Computer Engineering at the Institute of Engineering, Purwanchal Campus entitled **"A MACHINE UNLEARNING SDK AND API PLATFORM FOR PRIVACY-COMPLIANT SYSTEM"** is our own work and has not been previously submitted to any university for any academic award.

We authorize the Institute of Engineering, Purwanchal Campus to lend this report to other institutions or individuals for the purpose of scholarly research.

DEBENDRA SHEKHAR GUPTA (PUR078BCT030)

DIPESH ACHARYA (PUR078BCT032)

JEEVAN KUMAR KUSHWAHA (PUR078BCT037)

PRIYANKA KUMARI MISHRA (PUR078BCT057)

March 31, 2026

RECOMMENDATION

The undersigned certify that they have read and recommended to the Department of Electronics and Computer Engineering for acceptance, a project entitled **"A Machine Unlearning SDK and API Platform for Privacy-compliant ML Systems"**, submitted by DEBENDRA SHEKHAR GUPTA,DIPESH ACHARYA,JEEVAN KUMAR KUSHWAHA and PRIYANKA KUMARI MISHRA in partial fulfillment of the requirement for the award of the degree of **"Bachelor of Engineering in Computer Engineering"**.

.....

Asst. Prof. Pukar Karki

Supervisor

Department of Electronics and Computer Engineering

Purwanchal Campus, Institute of Engineering, Tribhuvan University

.....

Prof. Subarna Shakya, (PhD)

External Examiner

Department of Electronics and Computer Engineering

Pulchowk Campus, Institute of Engineering, Tribhuvan University

.....

Asst. Prof. Manoj Kumar Guragain

Head of Department

Department of Electronics and Computer Engineering

Purwanchal Campus, Institute of Engineering, Tribhuvan University

31st March, 2026

ACKNOWLEDGEMENT

We express our sincere gratitude to the Department of Electronics and Computer Engineering, IOE, Purwanchal Campus, Dharan, for providing us the valuable opportunity to undertake this project, which allowed us to apply and enhance knowledge and skills acquired during our Bachelor of Computer Engineering studies.

We are especially thankful to our supervisor, Ass. Prof. Pukar Karki, for his invaluable guidance, constructive feedback, and continuous support throughout the project. We also extend our sincere appreciation to the Head of Department, Asst. Prof. Manoj Kumar Guragain, for his encouragement and support.

We would like to express our heartfelt thanks to all the teachers, and staff of the department for their guidance, mentorship, and assistance, which have been instrumental in supporting our academic and project journey.

Debendra Shekhar Gupta PUR078BCT030
Dipesh Acharya PUR078BCT032
Jeevan Kumar Kushwaha PUR078BCT037
Priyanka Kumari Mishra PUR078BCT057

ABSTRACT

This project develops a practical machine unlearning system designed to address privacy compliance and selective data deletion challenges within AI/ML workflows. Leveraging the SISA (Sharding, Isolating, Slicing, and Aggregating) framework for localized retraining, the system enables efficient removal of specific training data points without requiring complete model retraining. The methodology follows an Agile development process, incorporating stakeholder analysis, modular system design, persistent metadata storage, append-only audit logging, and idempotent unlearning behavior for operational safety. Key implementations include support for any scikit-learn-compatible estimator (e.g., Linear Regression, Random Forests, SVM), configuration-driven deployment with environment overrides, optional non-blocking background synchronization to a monitoring dashboard, and persistent cross-run model history for traceability. Performance benchmarks on datasets ranging from 5K to 50K samples demonstrate significant speedup factors over full retraining, with accuracy retention analyzed across various shard configurations. Challenges addressed include hardware constraints, dataset availability for realistic removal scenarios, and adapting unlearning mechanisms to convolutional architectures such as ResNet. The result is a reusable SDK that supports both local-only and cloud-enabled privacy-compliant deployments, with remaining work focused on extended model support, additional dataset validation, and final deployment pipeline completion.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF ABBREVIATIONS

API	: Application Programming Interface
Colab	: Colaboratory
SDK	: Software Development Kit
GDPR	: General Data Protection Regulation
MLOps	: Machine Learning Operations
SISA	: Sharded,Isolated, Sliced and Aggregated
CCPA	: California Consumer Privacy Act
AI	: Artificial Intelligence
ML	: Machine Learning
CNN	: Convolutional Neural Network
MLP	: Multi-Layer Perceptron
CSV	: Comma-Separated Values
IEEE	: Institute of Electrical and Electronics Engineers
NeurIPS	: Neural Information Processing Systems
ICML	: International Conference on Machine Learning
arXiv	: arXiv e-print archive
UCI	: University of California, Irvine
CVPR	: Conference on Computer Vision and Pattern Recognition
CVF	: Computer Vision Foundation

CHAPTER 1

INTRODUCTION

1.1 Background

Machine learning models have become deeply integrated into everyday applications, from personalized content recommendations to critical tasks like medical diagnosis and financial fraud detection. These models learn patterns from vast amounts of user data, improving their accuracy and usefulness over time. However, this dependence on user data creates a fundamental tension with privacy rights. Regulations such as the General Data Protection Regulation (GDPR) in Europe and the California Consumer Privacy Act (CCPA) in the United States now establish clear legal requirements: users have the right to request deletion of their personal data from systems that process it. The technical challenge lies in how to honor such deletion requests when the data has already been incorporated into a trained machine learning model. Traditional approaches require complete model retraining on the remaining dataset, a process that becomes prohibitively expensive as model complexity and dataset sizes grow. This gap between legal requirements and technical capability has given rise to the field of machine unlearning, which aims to efficiently remove the influence of specific data points from trained models without full retraining.

1.2 Gap Identification

Despite growing research interest in machine unlearning, the practical adoption of these techniques remains limited. Academic papers propose sophisticated algorithms, but they often remain as theoretical prototypes or are tightly coupled to specific model architectures. Open-source libraries that do exist tend to focus on narrow use cases or require significant expertise to integrate into existing machine learning pipelines. For most developers and organizations, there is no straightforward, production-ready solution that allows them to build privacy-compliant systems with unlearning capabilities from day one. This project addresses that gap by developing PyUnlearn, a practical SDK and API platform that abstracts away the complexity of machine unlearning, making it accessible to developers working on real-world applications that

must comply with privacy regulations.

1.3 Motivation

The motivation for this project stems from a simple but increasingly urgent question: as users entrust their data to machine learning systems, can they also trust that their presence can be removed when requested? Current practices often treat data Qdeletion as a database operation, ignoring the fact that the model itself retains implicit knowledge of the deleted data. This disconnect between what users expect and what systems actually deliver creates legal risk for organizations and erodes user trust. Rather than treating privacy as an afterthought or a compliance checkbox, we believe that unlearning capabilities should be built directly into the machine learning development workflow. PyUnlearn represents an effort to make this possible, providing developers with tools that integrate naturally with existing workflows while ensuring that privacy guarantees are maintained throughout the model lifecycle.

1.4 Objectives

The project aims to achieve the following objectives:

- Develop an efficient selective unlearning framework that allows developers to integrate unlearning capabilities into their machine learning pipelines, with the flexibility to run locally or leverage cloud infrastructure for training and unlearning operations.
- Ensure that unlearning operations provide meaningful privacy guarantees while preserving model performance on retained data, balancing the trade-off between computational efficiency and model accuracy.
- Deliver a practical solution that enables compliance with GDPR, CCPA, and similar privacy regulations by providing auditable unlearning operations, persistent metadata tracking, and idempotent deletion semantics suitable for production deployment.

1.5 Scope and Limitations

This project focuses on implementing a practical machine unlearning SDK based on the SISA (Sharded, Isolated, Sliced, Aggregated) framework, targeting tabular datasets and machine learning models compatible with the scikit-learn ecosystem. The system supports both local deployment and cloud-enabled monitoring through an optional dashboard. While the approach demonstrates significant efficiency gains over full retraining, current limitations include the lack of support for deep learning architectures such as convolutional neural networks, reliance on a fixed sharding strategy that may not adapt optimally to all dataset distributions, and the absence of formal certification guarantees for the unlearning process. These limitations are acknowledged as areas for future extension beyond the scope of the current project.

CHAPTER 2

RELATED THEORY

2.1 Machine Unlearning

Machine unlearning refers to a class of methods that remove the influence of specific training records from a trained model while preserving utility on non-removed data. The motivation is both regulatory and operational: privacy regulations such as the General Data Protection Regulation (GDPR) and the California Consumer Privacy Act (CCPA) establish the *Right to be Forgotten*, giving users the legal right to request deletion of their personal data. Beyond compliance, practical machine learning pipelines frequently require removal of mislabeled, outdated, or sensitive samples after a model has already been deployed.

In conventional machine learning, deletion is typically handled by retraining the model from scratch on the remaining dataset. While this approach is conceptually simple and guarantees complete removal of the target data's influence, it becomes impractical as models grow in size and datasets expand. Retraining a large model could take hours or days, making it impossible to honor deletion requests within reasonable timeframes, especially when multiple deletion requests arrive frequently.

Beyond computational efficiency, a practical unlearning system must address two additional concerns. First, *verifiability* asks whether we can be confident that the target record no longer influences the model's predictions. Second, *auditability* requires the system to maintain a clear, tamper-resistant record of deletion requests and their outcomes. In real-world deployments, this demands persistent bookkeeping structures that track active sample counts, shard assignments, and unlearning history, ensuring that model state remains consistent across repeated runs and system restarts.

2.2 SISA Framework for Efficient Unlearning

The Sharded, Isolated, Sliced, and Aggregated (SISA) framework, introduced by Bourtoule et al., provides a structured approach to efficient machine unlearning by decomposing training into smaller, independently retrainable components. The core idea is straightforward: instead

of training one monolithic model on the entire dataset, the dataset is divided into multiple independent *shards*. Each shard is trained separately, producing its own sub-model. When a prediction is needed, the outputs from all shard models are aggregated, typically through majority voting for classification or averaging for regression.

The key insight for unlearning is that any training record belongs to exactly one shard. Therefore, when a deletion request arrives, only the shard containing that record needs to be retrained, leaving all other shards untouched. This localized retraining dramatically reduces computational overhead compared to full model retraining. The framework can be extended further by dividing each shard into time-ordered *slices*, enabling incremental learning where only the affected slice within a shard requires retraining.

From an engineering perspective, implementing SISA in a production system introduces practical requirements beyond the core algorithm. Persistent metadata storage is essential to maintain stable mappings from data indices to shards across system restarts. An append-only unlearning history log provides auditability by recording each deletion action with timestamps, identifiers, and reason codes. Idempotent unlearning behavior ensures that submitting the same deletion request multiple times does not corrupt model state or cause redundant retraining. These system-level considerations form the foundation of our implementation approach.

2.3 Model Architectures Supporting Machine Unlearning

The applicability of machine unlearning techniques varies across different model families, and the cost of unlearning depends heavily on how a model's training procedure can accommodate targeted retraining. For classical machine learning on structured data, linear models and tree-based ensembles are particularly well-suited to SISA-style sharding. These models are widely used in domains such as credit risk assessment, fraud detection, and income prediction, where both interpretability and regulatory compliance are important. The training process for these models is relatively stable, and retraining a shard can be performed independently without complex dependency management.

For neural network architectures, the landscape is more varied. Multi-Layer Perceptrons (MLPs) remain a common choice for tabular data and can be integrated into sharded training workflows with careful design. Convolutional neural networks (CNNs), such as the ResNet family, are

prevalent in computer vision tasks but present additional challenges. Their training involves complex optimization dynamics and dependencies across layers, making localized retraining less straightforward to implement without careful consideration of gradient propagation and initialization strategies.

Regardless of the specific architecture, the core requirement remains consistent: unlearning must enable removal of a record's influence by retraining only the minimally affected components rather than the entire model. This project focuses primarily on scikit-learn compatible estimators for classical machine learning tasks, establishing a solid foundation that can be extended to neural architectures in future work. The theoretical principles discussed in this chapter directly inform the system design decisions, particularly around data partitioning, persistence, and audit logging.

CHAPTER 3

LITERATURE REVIEW

Machine learning (ML) is a key technology across industries from personalized recommendations to healthcare diagnostics and fraud detection. These models rely on user data to improve predictions, raising privacy concerns. Regulations like GDPR and CCPA enforce a ”right to be forgotten,” allowing users to request deletion of their data from systems, including ML models. However, removing specific users' influence from a trained model is not straightforward. Most ML pipelines are designed for learning, not unlearning. Retraining an entire model from scratch for every deletion request is time-consuming, computationally expensive, and often impractical. This creates a technical gap for efficient user data forgetting.

Research in machine unlearning proposes methods like dataset partitioning and influence estimation. The SISA (Sharded, Isolated, Sliced Aggregation) framework is notable for balancing performance with deletion support. Yet, many methods remain academic prototypes or are tailored to specific models. There is a lack of general-purpose, developer-friendly libraries for machine unlearning in real-world scenarios. To address this, we introduce a Python package to help developers train models supporting future unlearning requests, focusing on user-level data tracking and integrating deletion as a core feature. The system is modular and compatible with standard scikit-learn estimators, offering a practical tool for privacy-aware ML.

In addition to algorithmic efficiency, recent practical deployments emphasize system-level requirements that are often under-addressed in the literature, such as persistence of model state across repeated executions, auditability of deletion actions, and operational monitoring. In particular, maintaining persistent metadata (e.g., index-to-shard mappings and active sample counts) and an unlearning history log supports verifiable and repeatable unlearning, while optional cloud-based monitoring can provide visibility into unlearning events and model statistics without altering the core ML workflow.

Researchers have explored different approaches. One well-known method, SISA, breaks data into smaller chunks and trains multiple isolated models. When a user requests deletion, only affected parts are retrained, making unlearning more efficient. We implement the SISA training paradigm introduced by Bourtoule et al., which breaks the training process into smaller components enabling localized retraining. The training pipeline starts with an annotated user-

centric dataset, where each record contains a user ID, label, and preprocessed feature vector. The dataset is split into s,

s shards using stratified sampling, where shard count s s is estimated by:

$$s = \sqrt{\frac{d \cdot n}{r}}$$

Here, d is the dataset dimension (number of features), n n the number of samples, and r r is a configurable parameter that controls the desired samples per shard. This formula helps balance the trade-off between retraining efficiency and model performance. Each shard can be further divided into time-ordered slices for more granular incremental training.

From a reproducibility standpoint, the sharded training design also enables persistent bookkeeping of shard composition and unlearning actions. Such bookkeeping is necessary to ensure that subsequent deletion requests (including repeated requests for the same record) can be handled deterministically and safely, and that unlearning outcomes can be inspected or summarized for evaluation.

Other methods include knowledge distillation, which involves retraining on a smaller cleaned dataset, and influence functions, which estimate each data point's effect on predictions. While promising, these remain complex for real-world use. Our modular design limits deletion impact to a subset of training, offering clear computational advantages over full retraining.

- Input: Annotated user dataset (e.g., CSV with user IDs, labels, features)
- Processing:
    1. Group data by user ID
    2. Split into s s shards via stratified sampling
    3. Divide each shard into m m time-ordered slices
    4. Initialize and train a base model per shard and slice, while recording persistent training metadata
- Output: Trained sub-models representing each shard-slice unit

This structure enables retraining only the slices containing data of users requesting deletion.

When a deletion request arrives, the system identifies the shard and slice(s) holding the user's data. The unlearning process involves:

- **Identification:** Use training metadata to locate the user's data within the sharded partitioning.
- **Data Removal:** Remove the user's samples from the affected partition; other partitions remain unchanged.
- **Localized Retraining:** Retrain only the affected component(s) (e.g., the corresponding shard and, where applicable, the relevant slice range), while unaffected shards remain intact.
- **Aggregation:** Reintegrate retrained components and combine predictions from all shards via voting or weighted averaging.
- **Audit Logging:** Record the unlearning action (timestamp, target identifier, affected partition, and stated reason) to support traceability across repeated executions.

Our framework's architecture supports user level deletion while minimizing retraining. The SISA Partitioner splits data, the Shard-Slice Trainer manages incremental training with checkpointing, and the Unlearning Engine handles deletion and retraining. The Evaluation module tracks accuracy, retraining time, and privacy metrics. This modular, scalable system enables efficient, privacy-preserving unlearning for real applications.

Localized retraining reduces computational cost, especially with frequent deletions, and ensures deletion consistency by fully removing user influence. Our design avoids shared gradients or centralized coordination during shard training, improving scalability and privacy.

While many methods remain academic, PyUnlearn fills the gap by providing a practical toolkit easy to adopt and integrate into ML workflows.

The literature demonstrates that while machine unlearning has gained significant research attention, the transition from theoretical frameworks to practical, production-ready tools remains incomplete. Existing approaches each offer distinct advantages—SISA provides computational efficiency through localized retraining, influence functions enable parameter-level updates, and certified removal offers theoretical guarantees—but no single solution comprehensively addresses the operational requirements of real-world deployment. PyUnlearn bridges this gap by

implementing the SISA framework with persistent state management, audit logging, and idempotent unlearning behavior, providing developers with a toolkit that is both algorithmically sound and practically usable. This work contributes to making privacy-compliant machine learning systems more accessible, aligning technical capability with regulatory expectations.

CHAPTER 4

METHODOLOGY

4.1 Project Development Method

The development of PyUnlearn follows an Agile methodology, enabling iterative progress, continuous feedback integration, and adaptive planning throughout the project lifecycle. This approach proved particularly suitable given the evolving nature of machine unlearning techniques and the need for frequent validation with real-world use cases.

4.1.1 Research

We begin by identifying the core challenges of machine unlearning in practical AI/ML workflows, particularly for privacy-focused deployments. Through stakeholder analysis and a review of privacy-driven operational constraints, we derive requirements that extend beyond algorithmic unlearning efficiency. These requirements include support for selective data deletion, privacy-compliant retraining, verifiable audit logging, persistence of model state across repeated executions, and compatibility with local as well as cloud-enabled monitoring environments. A systematic literature review was conducted to understand existing approaches and their limitations.

4.1.2 Strategize

Based on insights from the research phase, we define a system strategy that prioritizes modularity, compliance, and integration ease. We select the SISA framework as the principal unlearning paradigm due to its localized retraining properties and suitability for repeated deletion requests. Key architectural decisions, including the choice of SQLite for persistent storage and RESTful API design for the cloud platform, were finalized during this phase. In addition, we plan for multi-environment usability by supporting a configuration-driven workflow, enabling users to run the system purely locally or to optionally enable cloud synchronization and monitoring without requiring changes to their core machine learning code.

4.1.3 Document Requirements

We document functional and technical specifications needed to guide system development. Functional specifications include training, prediction, and unlearning operations, along with the ability to process single and batch deletion requests. Technical specifications include persistent metadata storage using SQLite, an append-only unlearning history log for auditability, and safe handling of repeated unlearning requests for the same record (idempotent behavior). We additionally specify configuration requirements to support user provided tokens and endpoint overrides when cloud synchronization is enabled. This step forms the blueprint for the subsequent system design and ensures all team members share a common understanding of deliverables.

4.1.4 System Design

We design the system architecture to support the complete machine unlearning workflow, including model management, metadata persistence, audit logging, and retraining orchestration. The pipeline ensures that each data deletion request is mapped to the minimal affected training partition, allowing retraining to be localized rather than applied to the entire model. All unlearning events are systematically recorded to support traceability and auditing. To enhance usability, the architecture also incorporates an optional synchronization layer for dashboard monitoring, implemented in a non-blocking manner so that background updates do not interfere with training or unlearning processes. This design enables the system to be packaged as a reusable SDK and deployed across diverse environments, from local development setups to production cloud infrastructure.

4.1.4.1 Sharded Training and Machine Unlearning Framework

The proposed framework adopts a sharded training strategy to enable efficient and scalable data removal. As illustrated in Figure 4.2, the dataset is initially partitioned into multiple independent shards to ensure data isolation. Each shard is trained separately to produce individual models, which are later aggregated into a unified predictive model. For finer granularity, each shard is further divided into smaller slices, allowing unlearning to be performed at either shard

level or slice level depending on the deletion scope. This hierarchical structure ensures that only the affected partition requires retraining when a deletion request is made, while the rest of the model remains unchanged. Consequently, the approach significantly reduces computational overhead and improves efficiency while maintaining overall model integrity. Additionally, this design supports parallel training across shards, enabling faster initial model development. The isolation between shards also provides natural privacy boundaries, as no data flows between partitions during training or unlearning operations. This makes the framework particularly suitable for deployment scenarios where data compartmentalization is required for regulatory compliance.

4.1.4.2 Metadata Storage for SISA-Based Unlearning

The effectiveness of the SISA framework depends critically on persistent metadata that captures the relationship between training data and the sharded model structure. Without proper metadata management, the system cannot reliably identify which shard contains a given data point when a deletion request arrives, nor can it maintain auditability across multiple unlearning operations.

Metadata Schema Design

The system implements a SQLite-based storage layer with the following core tables:

- **models:** Stores global model metadata including model ID, creation timestamp, dataset hash, total shard count, aggregation strategy (voting or averaging), and the scikit-learn estimator type used for training.
- **shards:** Maintains per-shard information such as shard ID, associated model ID, shard index, number of active samples, file path to the pickled model artifact, and timestamp of the last retraining operation.
- **index_mapping:** The critical mapping table that links each original data point index to its assigned shard and slice. This table enables O(1) lookup when processing deletion requests, avoiding the need to scan the entire dataset.
- **unlearning_history:** An append-only audit log recording each unlearning event with

request ID, timestamp, target data point indices, affected shard IDs, reason code (e.g., 'GDPR_REQUEST', 'DATA_CORRUPTION'), and operation status.

Metadata Lifecycle During Unlearning

When a deletion request arrives, the system follows this metadata-driven workflow:

1. **Lookup:** Query the index_mapping table to retrieve the shard and slice containing each target data point.
2. **Validation:** Verify that the target indices have not been previously deleted by checking the unlearning_history table, ensuring idempotent behavior.
3. **Isolation:** For each affected shard, load the current training data, remove the target samples, and update the active sample count in the shards table.
4. **Retraining:** Retrain the affected shard model and update its file path and last_retrain_timestamp in the shards table.
5. **Logging:** Insert a new record into the unlearning_history table with complete details of the operation.
6. **Commit:** All database operations are wrapped in a transaction, ensuring that partial updates do not occur if any step fails.

Persistence and Portability

All metadata is stored in a local SQLite database file alongside the trained model artifacts. This design ensures that the entire system state—models, shard assignments, and audit logs—can be moved, backed up, or restored as a single unit. When cloud synchronization is enabled, metadata snapshots are periodically pushed to the monitoring dashboard, providing visibility without requiring access to the local storage.

Auditability and Compliance

The append-only nature of the unlearning history table is particularly important for regulatory compliance. Each deletion request is permanently recorded with a timestamp and reason

code, creating an immutable record that can be presented during audits to demonstrate that user deletion requests have been properly processed. The persistent index mapping also enables verification that no deleted data remains active in any shard model.

4.1.5 Develop

Using Agile sprints of two-week duration, we implement the system in modular increments. Each sprint delivers SDK features such as SISA-based training and aggregation, persistent metadata management, and unlearning operations with explicit reason tracking. Feature development also includes configuration management (e.g., YAML- and environment-based overrides) and optional background synchronization to a monitoring service when enabled by the user. Code reviews and pair programming sessions were conducted to maintain code quality and knowledge sharing across team members. This incremental approach allows validation of correctness after each feature addition, particularly for persistence and idempotent unlearning behavior.

4.1.6 Integration and Testing

Comprehensive testing is conducted to validate system integrity and unlearning behavior. Unit and integration tests verify correct shard construction, consistent index-to-shard mapping, and correctness of localized retraining. Additional validation focuses on persistence and auditability, including the ability to reload model state across multiple runs, to append unlearning actions to the history log, and to safely ignore repeated unlearning requests for already-removed indices without raising failures. Performance testing was conducted to benchmark unlearning latency across varying dataset sizes and shard configurations. Where cloud monitoring is enabled, synchronization is tested to ensure that model statistics and history snapshots can be propagated without requiring explicit user-side synchronization calls.

4.1.7 Deploy

We prepare the system for deployment by releasing stable SDK versions through standard packaging workflows (PyPI distribution), alongside examples and quickstart templates. Deployment

considerations include configuration distribution (project-level configuration files and environment variable overrides) to securely provide cloud tokens and endpoint configuration when the dashboard is used. The dashboard backend (FastAPI) and frontend (React) are deployed as separate services, enabling users to run the monitoring stack locally during development or to integrate it into a cloud environment as required. Containerization using Docker is supported for consistent deployment across environments.

4.1.8 Support / Maintenance

Post-deployment, we onboard early users to gather real-world feedback and to evaluate usability under realistic deletion-request scenarios. Based on this feedback, we iteratively improve model support, user experience, and documentation quality. Maintenance activities also include monitoring reliability of persistence and logging mechanisms, ensuring backward compatibility of stored metadata where feasible, and refining configuration defaults to reduce integration effort for both local-only and cloud-enabled deployments. A public issue tracker is maintained to manage feature requests and bug reports from the user community.

4.2 Project Timeline

The project development is structured across major milestones spanning the academic year, with completed work consolidated into the earlier phases and remaining activities scheduled through the end-of-March submission period. Key milestones included algorithm selection in September, core SDK implementation from October to December, dashboard development in January, integration testing in February, and final documentation and deployment preparation in March.

Figure 4.3: Project Timeline Snapshot (Gantt Chart Representation)

Figure 4.1: Phases of the Agile Development Methodology

Figure 4.2: Sharded Training and Machine Unlearning Architecture

CHAPTER 5

RESULT AND DISCUSSION

5.1 Work Completed

- **Comprehensive Algorithm Analysis:** Conducted a detailed study of the SISA unlearning algorithm using the Adult Income dataset, evaluating sharding, retraining, and unlearning effects.
- **Broader Algorithm Support:** Implemented the SDK to work with any scikit-learn compatible estimator, enabling training and unlearning workflows for classical machine learning algorithms such as Linear Regression, Logistic Regression, Ridge Regression, Support Vector Machines, Random Forests, and Gradient Boosting models.
- **End-to-End Dashboard Integration:** Implemented the complete backend frontend connection for the monitoring dashboard, enabling the interface to fetch and visualize model runs and unlearning events through API endpoints.
- **Persistent History and Auditability:** Introduced persistent storage for model history and unlearning metadata such that re-running experiments does not overwrite prior runs. This improves traceability and supports reproducible reporting.
- **Robust Unlearning Behavior:** Added idempotent handling for repeated unlearning requests on the same training indices (i.e., repeated "forget" operations do not corrupt state), improving operational safety.

- **Configuration File Design:** Added a configuration-driven setup to define variables and API keys with support for environment overrides, allowing deployments to avoid hard-coded secrets.
- **Background Synchronization:** Implemented optional automatic, non-blocking background synchronization so that model snapshots and history are pushed to the dashboard without requiring explicit user code.

Figure 5.1: Main dashboard showing unlearning status and metrics

5.1.1 Implementation and Performance Evaluation

Figure 5.2 illustrates the configuration setup used in the system, where API keys and environment-specific parameters are defined. This configuration-driven approach enables flexible deployment across local and cloud environments.

Figure 5.2: Configuration file setup with API key

The system was implemented with a focus on modularity, scalability, and performance opti-

mization. Key implementation highlights are summarized below:

- **Folder Structure and Package Organization:** The codebase was organized into a modular and standardized structure, enabling easy scalability, maintainability, and deployment across different environments.
- **Performance Benchmarking:** Extensive benchmarking was conducted to compare SISA-based unlearning with full model retraining across varying dataset sizes (5K to 50K samples) and different shard configurations. These experiments highlight the trade-offs between shard granularity, unlearning latency, and model performance.

Figure 5.3: Impact of shard count on unlearning latency in the SISA framework

Figure 5.4: Accuracy retention across different shard configurations

Figure 5.5: Speedup factor of SISA unlearning compared to full retraining across dataset sizes

5.2 Challenges Faced

During the development and evaluation of the system, several challenges were encountered:

- **Technical Limitations:** Hardware constraints significantly impacted large-scale model training and repeated unlearning cycles, leading to increased computation time.

- **Dataset Availability:** Obtaining diverse and high-quality datasets that accurately simulate real-world data deletion scenarios proved to be challenging.
- **Model-Specific Constraints:** Adapting the SISA framework to complex architectures such as convolutional neural networks (e.g., ResNet) introduced additional implementation complexity.
- **Operational Robustness:** Ensuring persistence, auditability, and safe handling of repeated unlearning requests required careful design of state management, serialization mechanisms, and cross-run consistency.

Bibliography

[1] L. Bourtoule, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot, "Machine unlearning," in *IEEE Symposium on Security and Privacy (SP)*, 2021.

[2] Y. Cao and J. Yang, "Towards making systems forget with machine unlearning," in *IEEE Symposium on Security and Privacy*, 2015.

[3] A. Ginart, M. Guan, G. Valiant, and J. Y. Zou, "Making AI forget you: Data deletion in machine learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[4] C. Guo, T. Goldstein, A. Hannun, and L. van der Maaten, "Certified data removal from machine learning models," in *International Conference on Machine Learning (ICML)*, vol. 119, 2020.

[5] C. Chen, F. Sun, M. Zhang, and B. Ding, "Recommendation unlearning," in *Proceedings of the Web Conference (WWW)*, 2022.

[6] A. Sekhari, J. Acharya, G. Kamath, and A. T. Suresh, "Remember What You Want to Forget: Algorithms for Machine Unlearning," *arXiv preprint arXiv:2103.03279*, 2021.

[7] A. Golatkar, A. Achille, and S. Soatto, "Eternal Sunshine of the Spotless Net: Selective Forgetting in Deep Networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[8] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership Inference Attacks against Machine Learning Models," in *IEEE Symposium on Security and Privacy (SP)*, 2017.

[9] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 (General Data Protection Regulation)," *Official Journal of the European Union*, L 119, pp. 1–88, 4 May 2016.

[10] D. Dua and C. Graff, "UCI Machine Learning Repository," University of California, Irvine, School of Information and Computer Sciences, 2019.

[11] A. Warnecke, L. Pirch, C. Wressnegger, and K. Rieck, "Machine unlearning of features and labels," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2021.

[12] S. Schelter, S. Grafberger, and T. Dunning, "Heterogeneous data management for machine learning: A survey," in *Proceedings of the VLDB Endowment*, vol. 14, no. 12, pp. 3324–3336, 2021.